Network Science

Lecture 04: Communities and Graph Partitioning

Prof. Dr. Michael Granitzer

Dr. Kanishka Ghosh Dastidar

Dr. Jelena Mitrovic

Speaker notes

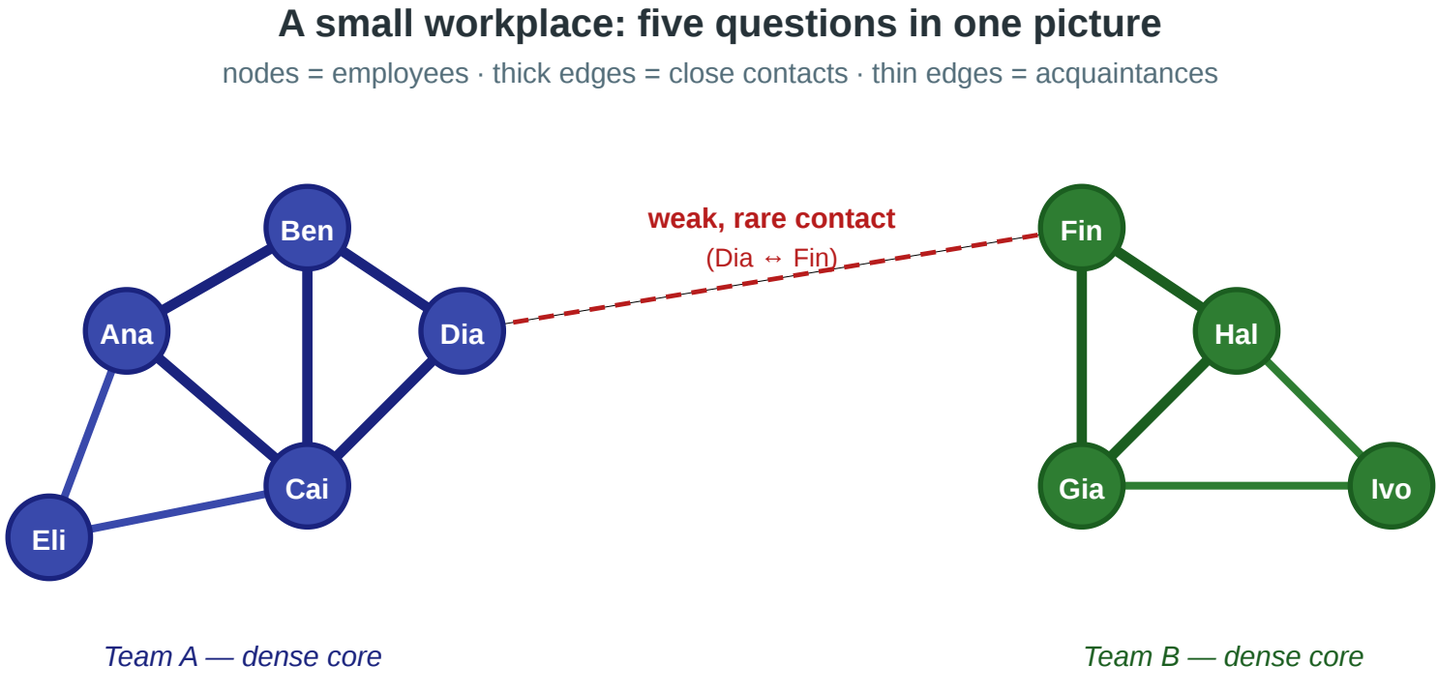
This lecture continues the modeling-vs-measurement arc from Lecture 03. In L03 we classified *edges* (strong vs. weak), discovered that the exact recovery problem (MaxSTC) is NP-hard, and fell back on polynomial-time structural proxies. In L04 we classify *nodes* (structural roles, centralities) and *sets of nodes* (communities), and we find exactly the same pattern: the theoretically ideal objective (modularity maximization) is NP-hard, and we fall back on heuristics. The whole course is held together by that single loop: theoretical construct → intractable recovery → polynomial-time proxy → empirical test.

From Classifying Edges to Classifying Nodes and Groups

In Lecture 03 we asked which *edges* carried novel information — strong vs. weak, bridges vs. embedded. Today we ask which *nodes* occupy important positions, and how the graph splits into dense groups.

Let us return to the same workplace.

The Same Scenario, New Questions



Questions we'd like to answer from this picture

① Will Eli and Ben become friends? ② How do we separate close from distant ties?
③ Which single edge controls all information flow between teams? ④ Where would a rumor spread fastest? ⑤ Which tie would we cut to isolate a leak?

Same graph as last time — two teams linked by a single weak Dia–Fin contact. Lecture 03 asked about the edges. Today the questions are about the nodes and the groups.

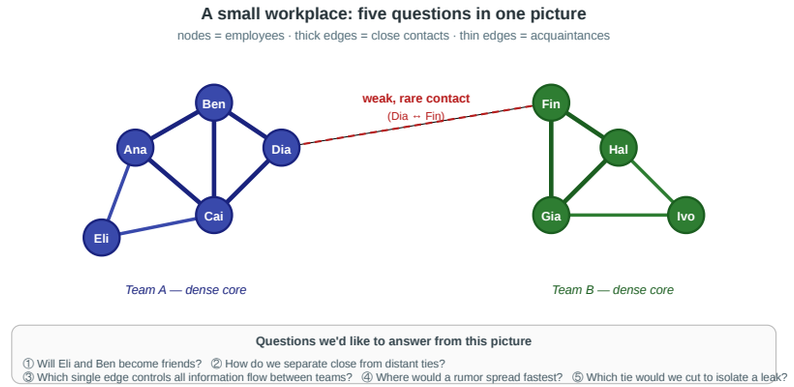
Speaker notes

Reusing the L03 scenario is deliberate. Students should feel that the lecture is continuing a single investigation rather than starting a new topic. The same figure will now reveal a different set of concepts depending on which question we ask.

Four Group-Level Questions

1. **Importance.** Who is the single most important person in this office — and what does "important" even mean?
2. **Boundaries.** Where is the line between team A and team B, and could an algorithm discover it *without being told the labels*?
3. **Uniqueness.** If we ran community detection on a 10 000-person company, would the answer be unique — or would different methods give different maps?
4. **Validation.** How do we know when a detected community is real rather than an artifact of the algorithm?

Every concept in today's lecture is a formal answer to one of these questions.



Speaker notes

Each question maps to a module: (1) structural roles and centrality, (2) communities and modularity, (3) divisive vs. agglomerative community-finding heuristics, (4) the Zachary karate club as an empirical anchor. Keep referring back to the workplace figure: Ana and Ben anchor the embedded role, Dia anchors brokerage, and the Dia–Fin edge anchors the community boundary.

Theory vs. Measurement — Continued

The gap we opened in Lecture 03 does not close in Lecture 04. Every concept today has a theoretical form we cannot compute on real data and a measurable proxy we can.

Question	Theoretical construct	Measurable proxy
Who is most important?	*(no single answer — multiple theories)*	Degree / closeness / betweenness / eigenvector
What makes a group a group?	Community with high internal density	Modularity Q
Best partition?	Global modularity maximum (NP-hard)	Girvan–Newman / Louvain heuristics
Is a partition meaningful?	Agreement with ground-truth labels	Benchmark datasets (Zachary's karate club)

Speaker notes

This table is L04's version of the L03 roadmap. The first row is slightly different from the others because centrality is not a single construct with a tractable/intractable split — it is a *family* of competing operationalizations, and the "measurement" column already reflects the theoretical pluralism. The second and third rows show the classical modeling/measurement split from L03, now applied to groups instead of edges.

Takeaway

L03 classified edges; L04 classifies nodes and groups. The modeling–measurement loop is the same: a theoretically clean objective is NP-hard, polynomial-time heuristics approximate it, and empirical benchmarks test whether the heuristics recover meaningful structure.

Speaker notes

This takeaway names the shape of the lecture before any content arrives. Refer back to it when introducing modularity ("this is the theoretical objective, and it is intractable to maximize exactly"), when introducing Girvan–Newman and Louvain ("these are the polynomial-time heuristics"), and when introducing Zachary's club ("this is the empirical test").

Roles and Social Capital

Guiding Question: What kinds of structurally different positions can people occupy inside the same network?

Speaker notes

The point of this module is to dismantle the intuitive notion that "important nodes" form a single category. A high-degree hub, an embedded clique member, and a structural broker are three distinct positions with different social and informational consequences. Students should leave the module able to name and contrast all three on the workplace figure.

Three Structural Positions

Embeddedness. A node whose neighbors are themselves densely interconnected. High clustering coefficient.

Brokerage. A node connecting otherwise separated groups. Low clustering coefficient, high betweenness.

Structural hole. The empty region in the network that a brokerage position spans.

They are related but not identical: **the hole is the missing connection between groups; the broker is the actor positioned across that missing connection.**

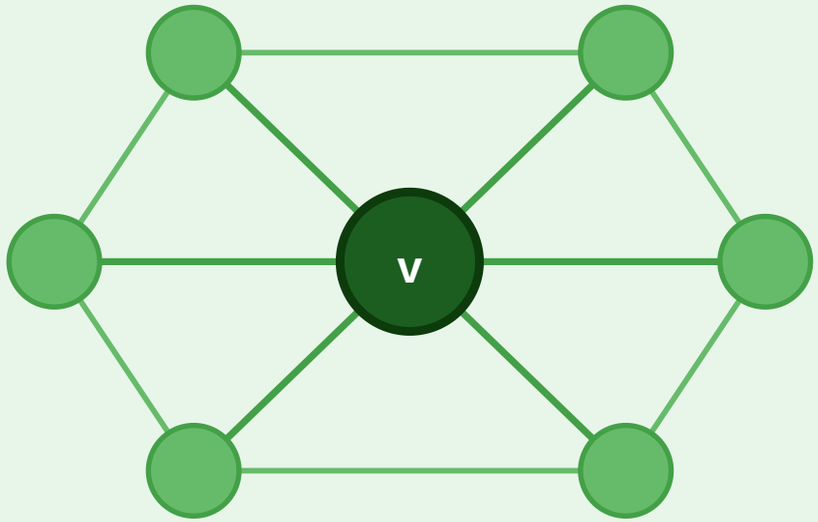
Speaker notes

Embeddedness is a property of a node, brokerage is a relational role, and a structural hole is the absence of edges that makes brokerage possible. This distinction is the likely confusion point: brokerage and structural holes are not synonyms. A structural hole is the gap; brokerage is the position or action of spanning the gap. In the workplace figure, Ana is embedded (her neighbors Ben and Cai are directly connected), Dia is a broker (the single cross-team tie runs through her), and the absence of direct Eli–Fin, Cai–Gia, Ben–Hal edges forms the structural hole between the teams.

Embedded vs. Broker

Embedded Node

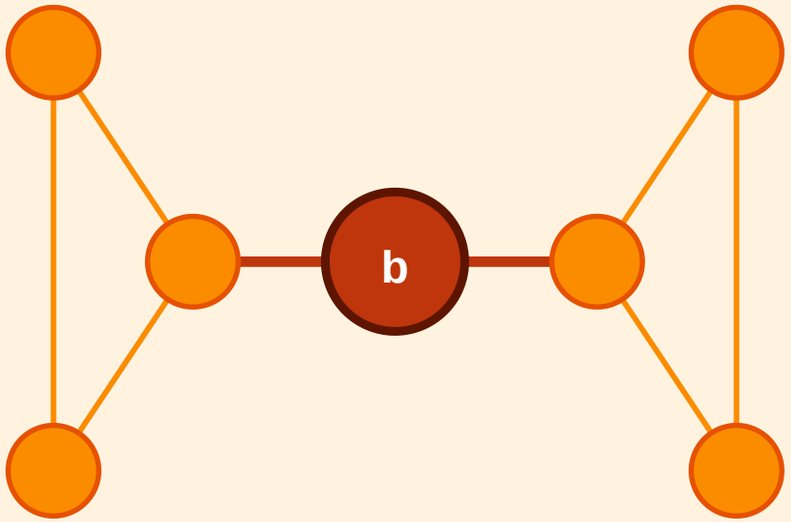
Dense, mutually connected neighbourhood



High clustering · trust · redundant info

Broker (Structural Hole)

Sole link between two separated clusters



Low clustering · high betweenness · novel info

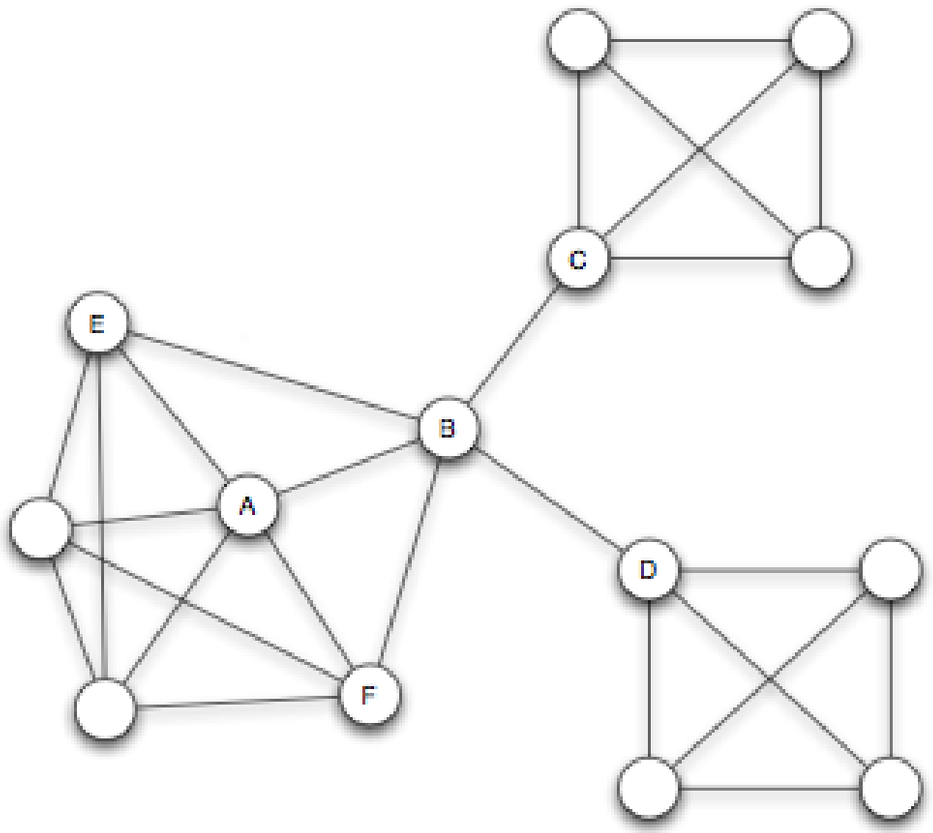
The embedded node v sits inside a tightly connected neighborhood — its removal barely affects connectivity. The broker b is the only short link between two clusters — its removal would isolate them.

Speaker notes

The visual contrast highlights how the same graph structure produces two opposite kinds of importance. The embedded node enjoys trust, redundancy, and reinforced norms; the broker enjoys early information access and gatekeeping power. Burt's empirical research showed that brokers are systematically over-rewarded in organizational settings — they earn more, get promoted faster, and produce more innovations — precisely because they capture the value of structural holes.

Structural Holes as Social Capital

A node that acts as the sole short link between otherwise separated groups spans a *structural hole* (Burt, 1992).



Source: Burt 1992

- **Information advantage:** the broker receives early, non-redundant signals from independent groups.
- **Control advantage:** the broker mediates flows between groups that cannot communicate directly.

Not the same thing: remove the missing gap and the structural hole disappears; remove the spanning actor and the brokerage position disappears.

Speaker notes

This is the social-capital interpretation of the formal structural-hole definition. The advantage evaporates the moment triadic closure reconnects the broker's contacts — which is why brokers in real organizations sometimes discourage their contacts from meeting directly.

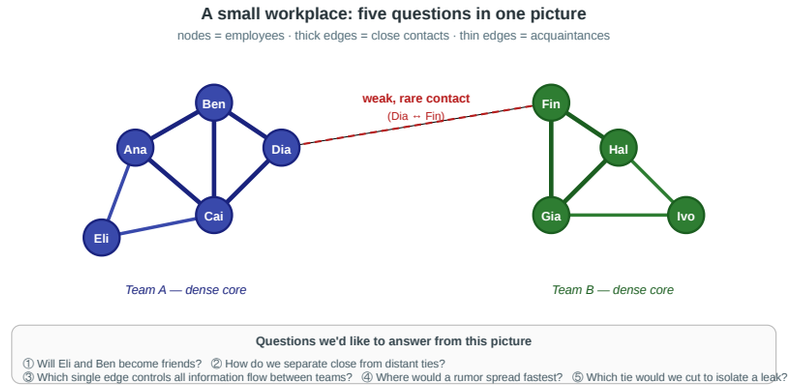
Takeaway

"Importance" is not one thing. Brokerage, embeddedness, and degree capture different social advantages — and the same node can sit in multiple roles depending on the question.

Quick Check

Look again at the workplace figure.

1. Which of these nine people is the clearest *broker*, and what structural feature makes them one?
2. Which is the most clearly *embedded*?
3. What would change structurally if Dia left the company tomorrow?



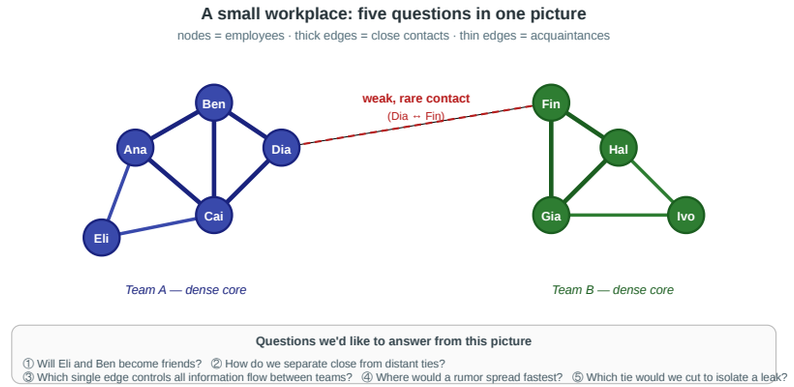
Speaker notes

Dia is the only broker: she is the only endpoint of the single cross-team edge. Ana, Ben, and Cai are all embedded inside the left triangle — each of them has directly connected neighbors. If Dia leaves, the only cross-team tie vanishes and the two teams become disconnected components — an extreme form of the structural-hole intuition. This quick check directly reuses the L03 scenario to make the new vocabulary concrete.

Quick Check

Look again at the workplace figure.

1. Which of these nine people is the clearest *broker*, and what structural feature makes them one?
2. Which is the most clearly *embedded*?
3. What would change structurally if Dia left the company tomorrow?



Solution: Dia is the broker because the cross-team edge runs through her. Ana, Ben, and Cai are embedded in the left triangle; Ben is strongest because his neighbors are mutually connected. If Dia leaves, the Dia–Fin edge disappears and the two teams become disconnected components.

Centrality

Guiding Question: How should we measure which nodes are important?

Speaker notes

Centrality is not a single quantity but a family of operationalizations, each formalizing a different intuition about importance. The choice of centrality measure is a modeling choice with consequences — picking degree when betweenness is the right concept will systematically miss the most structurally critical nodes.

Centrality

Guiding Question: How should we measure which nodes are important?

$$\text{importance} \neq \text{degree alone}$$

Degree Centrality — Local Exposure

$$C_D(v) = \frac{\deg(v)}{n-1}$$

Raw form. $C_D^{raw}(v) = \deg(v)$

Meaning. How many direct contacts does v have?

Theory. Direct exposure: importance comes from immediate contacts.

Range. Raw: 0 to $n-1$; normalized: 0 to 1.

Complexity. All scores: $O(n+m)$ including initialization; sorted ranking adds $O(n \log n)$.

Example. In a village gossip network, high-degree people hear from many neighbors directly.

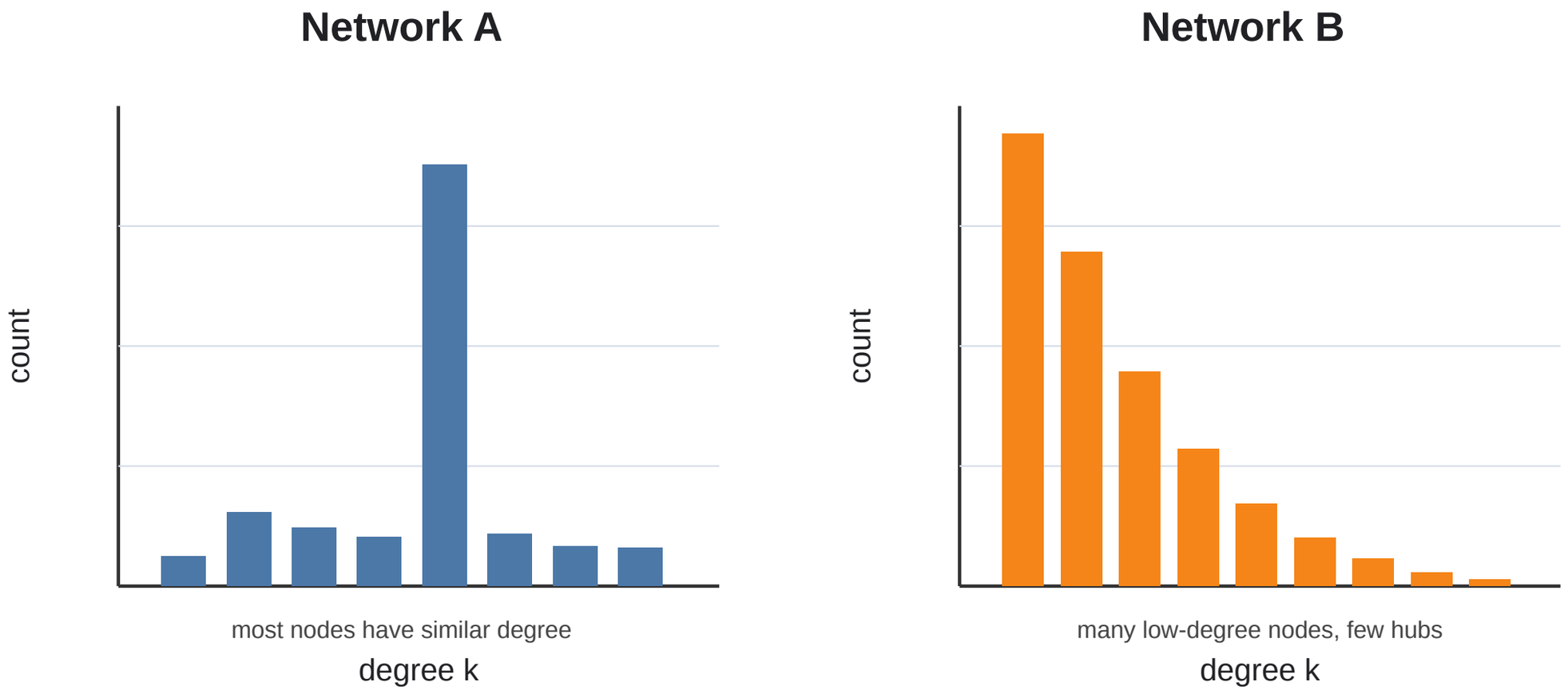
Network diagnostic. The **degree distribution** is a quick property check: equal-sized degrees suggest homogeneous structure; a heavy-tailed / power-law-like distribution suggests hubs and inequality (Barab'asi & Albert 1999).

Speaker notes

Degree is the simplest centrality and the easiest to compute. It is also the easiest to over-interpret. It measures direct exposure, not global position. Use it when direct contacts are the mechanism: gossip, local recruitment, direct infection exposure. The degree distribution is often the first descriptive statistic of a network: if almost all nodes have similar degree, degree centrality will not strongly differentiate roles; if the distribution is heavy-tailed, hubs dominate many processes.

Use of Degree Centrality — Degree Distribution

Show the degree distribution either as (i) histogram (below) or (ii) sorted by degree count to highlight distributional properties.



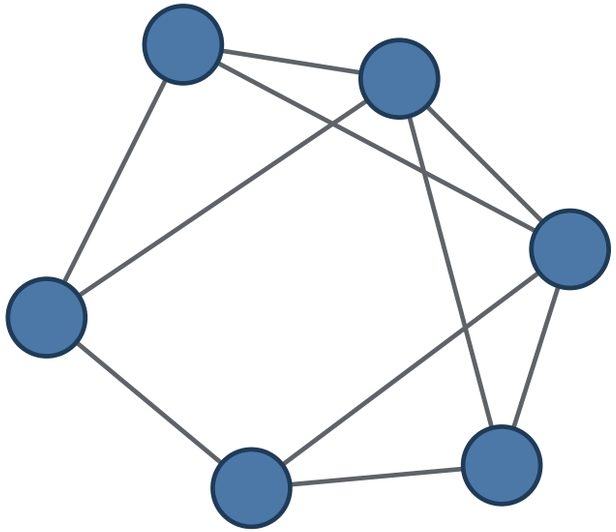
Make a guess: **what could a network with this degree distribution look like?**

Speaker notes

This is the right advanced use for degree because degree centrality is local but the distribution of all degrees is a global network fingerprint. A narrow distribution suggests equality of local opportunity and no obvious hub class. A heavy-tailed distribution suggests hub inequality, vulnerability to targeted hub removal, and strong differences between average and typical nodes.

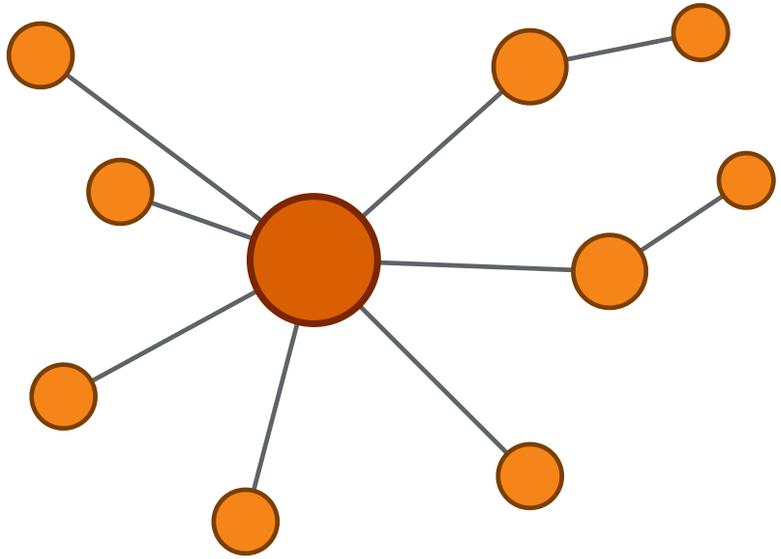
Degree Distribution — Possible Graphs

A: homogeneous



similar degree, no obvious hub

B: hub-dominated



few hubs, many peripheral nodes

These are not unique reconstructions. Many graphs can share the same degree distribution, but the distribution gives a useful first guess about whether the network is homogeneous or hub-dominated.

Speaker notes

This slide resolves the guessing activity without pretending that a degree distribution uniquely determines the network. The pedagogic point is modest: distributions are structural fingerprints, not full graph descriptions.

Closeness Centrality — Short-Path Access

$$C_C(v) = \frac{n-1}{\sum_{u \neq v} d(v,u)}$$

Distance input. Shortest-path distances $d(v,u)$ from v to all other nodes.

Meaning. How near is v to all other nodes on average?

Theory. Accessibility: importance comes from short paths to everyone.

Range. Normalized: 0 to 1; 1 only if v is adjacent to everyone.

Complexity. One node: $O(n+m)$ BFS; all nodes: $O(n(n+m))$ unweighted.

Example. An emergency-response hub with low average travel distance to all districts has high closeness.

Use. Closeness appears in accessibility studies: which hospital, station, or facility minimizes network distance to a population center

Speaker notes

Closeness is a global measure based on the shortest-path metric. Its theoretical construct is accessibility: a node is central if it can reach the entire graph with few steps. This is appropriate for diffusion or service placement when every target matters. It can be misleading in disconnected graphs unless harmonic closeness is used, because unreachable nodes make the distance sum ill-defined.

Harmonic Centrality — Robust Reachability

$$H(v) = \sum_{u \neq v} \frac{1}{d(v,u)} \quad \text{with } \frac{1}{\infty} = 0$$

Reciprocal-distance idea. Replace distances by $1/d(v,u)$ before summing.

Meaning. Many reachable nodes through short paths.

Theory. Robust accessibility: unreachable nodes contribute 0, not ∞.

Range. 0 to $n-1$; $n-1$ means direct access to everyone.

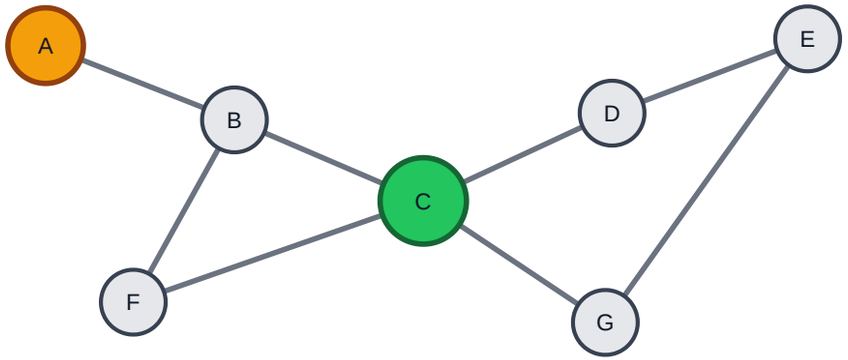
Complexity. One node: $O(n+m)$ BFS; all nodes: $O(n(n+m))$ unweighted.

Harmonic centrality is the disconnected-graph extension of closeness: unreachable nodes contribute 0 instead of making the distance sum ill-defined.

Speaker notes

Harmonic centrality belongs directly after closeness because it fixes the main practical problem of ordinary closeness. It preserves the accessibility intuition, but replaces the arithmetic treatment of distances with reciprocal distances. This makes it robust when a network has disconnected components, while keeping the same shortest-path computational burden.

Use of Closeness Centrality — Accessibility



C is a better service hub: short paths to almost every district.
A is connected, but peripheral: average distance is larger.

Question. Where should we place a service hub so it can reach everyone quickly?

Workflow.

1. Model roads, rail, or communication links as weighted edges.
2. Compute closeness or harmonic closeness.
3. Compare candidate locations against demand or population.

Class prompt: ask which node is the best emergency hub, then ask what changes if one district becomes disconnected.

Speaker notes

This is the advanced closeness use case I recommend: accessibility and facility location. It prepares the need for harmonic centrality later: ordinary closeness is fragile when the graph is disconnected.

Betweenness Centrality — Brokerage and Control

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

σ_{st} is the number of shortest s-t paths; $\sigma_{st}(v)$ counts those passing through v.

Meaning. How often does v sit between other nodes?

Theory. Brokerage: importance comes from sitting on paths between others.

Range. Normalized: 0 to 1; 1 means all shortest paths between others use v.

Complexity. Exact Brandes: $O(n(n+m))$ unweighted; weighted graphs are slower.

Difference from closeness. Closeness asks: "How near is v to everyone?" Betweenness asks: "How many shortest paths between *other* nodes depend on v?"

Exact betweenness for one focal node is not just one BFS: it still needs shortest-path dependency information across many source-target pairs. Brandes computes all nodes for essentially the same asymptotic cost.

Speaker notes

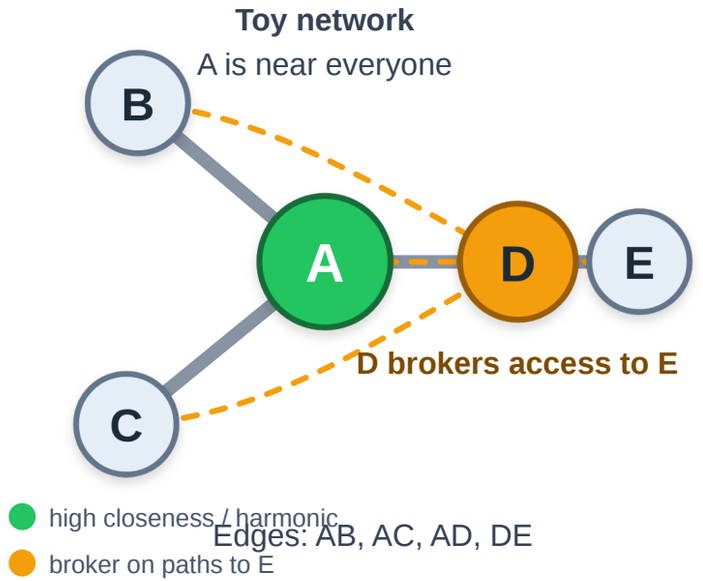
Betweenness formalizes brokerage as shortest-path control. It is the centrality measure most directly connected to the structural-hole idea: if groups communicate mainly through shortest paths, a high-betweenness node can mediate or block flows. The weakness is the assumption that traffic follows shortest paths. In many real systems, flows are random, strategic, or capacity-limited, so betweenness is a model of control rather than a universal truth.

Computing Closeness, Harmonic, and Betweenness

Toy network: A is connected to B, C, D; D is connected to E. Here $n = 5$ and normalized betweenness divides by $\binom{4}{2} = 6$ possible pairs among the other nodes.

Node	Distances	Closeness	Harmonic $H(v) = \sum 1/d$	Harmonic mean distance
A	$1, 1, 1, 2$	$4/5 = 0.80$	3.50	$4/3.50 = 1.14$
D	$1, 1, 2, 2$	$4/6 = 0.67$	3.00	$4/3.00 = 1.33$
E	$1, 2, 3, 3$	$4/9 = 0.44$	2.17	$4/2.17 = 1.85$

Node	Shortest paths between others that pass through node	Betweenness
A	5 of 6	$5/6 = 0.83$
D	3 of 6	$3/6 = 0.50$
E	0 of 6	0

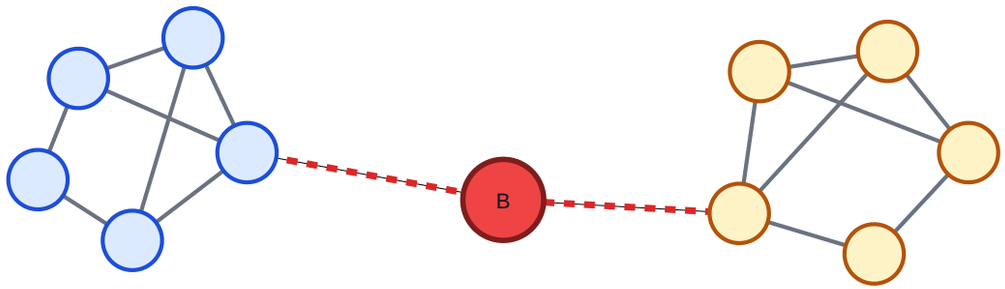


A has the highest closeness and harmonic centrality because it minimizes distance. D is less close but still has brokerage value because paths from B or C to E must pass through it.

Speaker notes

Compute the same toy graph three ways. For closeness, focus on the sum of distances from the focal node to everyone else. For harmonic centrality, sum reciprocal distances; the harmonic mean distance is the inverse-style average $(n-1)/H(v)$. For betweenness, ignore paths that start or end at the focal node and count only shortest paths between other pairs. This makes the distinction concrete: closeness and harmonic centrality measure accessibility; betweenness measures dependency or brokerage.

Use of Betweenness Centrality — Vulnerability



The red broker lies on most shortest paths between clusters.
Remove it: the largest connected component shrinks sharply.

Proposal. Use betweenness as a stress-test tool.

1. Compute node or edge betweenness.
2. Remove the top-ranked nodes/edges.
3. Track largest connected component size.
4. Compare against random removal.

If targeted removal fragments the graph much faster than random removal, the network depends on a small set of brokers or bridges. For large graphs, approximate by sampling source nodes.

Speaker notes

This is the cleanest advanced betweenness activity because it connects the formula to an observable consequence: fragmentation. It works for transport, supply chains, communication networks, and organizations. It also gives a reason to discuss computational cost: exact betweenness is expensive, so large graphs often use sampled or approximate betweenness.

Eigenvector Centrality — Recursive Prestige

Node equation.

$$C_E(v) = \frac{1}{\lambda} \sum_u A_{vu} C_E(u)$$

Vector equation.

$$Ax = \lambda x$$

Meaning. Important nodes are connected to important nodes.

Theory. x is the leading eigenvector of A; this is recursive prestige (Bonacich 1987).

Range. Scores depend on normalization, often 0 to 1.

Complexity. Power iteration costs $O\big(k(n+m)\big)$ for k iterations, often written $O(km)$ on connected sparse graphs:

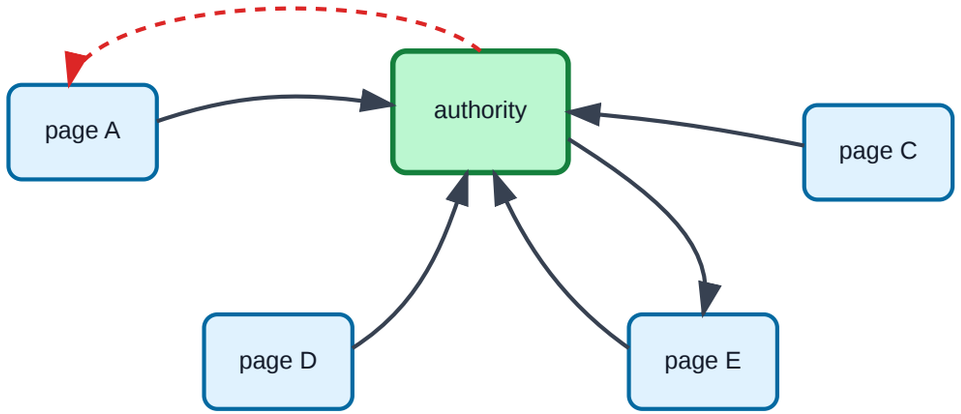
$$x^{(t+1)} = \frac{Ax^{(t)}}{\|Ax^{(t)}\|}$$

The two equations are the same idea: the score of each node equals the weighted prestige of its neighbors; in vector form this is the eigenvector equation.

Speaker notes

Eigenvector centrality moves from counting contacts to weighting contacts by their own importance. That is why it captures prestige and status more naturally than degree. The node equation and the vector equation are the same claim written at different levels. The theoretical construct is a fixed point: centrality equals a weighted sum of neighbors' centralities. Power iteration approximates that fixed point by repeated sparse matrix-vector multiplication. PageRank adds direction, damping, and random jumps to make this stable on the Web's directed graph.

Use of Eigenvector Centrality — PageRank



Follow links with probability alpha; randomly jump with probability 1-alpha.
Power iteration estimates the long-run visit probability of each page.

$$PR(v) = \frac{1-\alpha}{n} + \alpha \sum_{u \to v} \frac{PR(u)}{\text{outdeg}(u)}$$

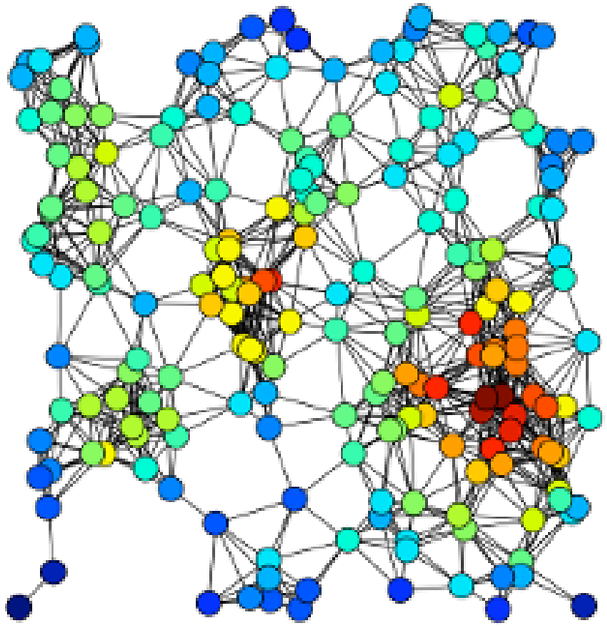
Random surfer. With probability α, follow a link. With probability $1 - \alpha$, jump to a random page.
Range. PageRank scores are probabilities: each score is in $[0, 1]$ and all scores sum to 1.
Power iteration. Repeatedly update all scores until they converge. Each iteration is sparse: $O(n + m)$, often written $O(m)$ on connected graphs.

PageRank scales well by power iteration, but it is cheatable: link farms manufacture endorsements. Harmonic centrality is harder to game that way, but exact computation is more expensive.

Speaker notes

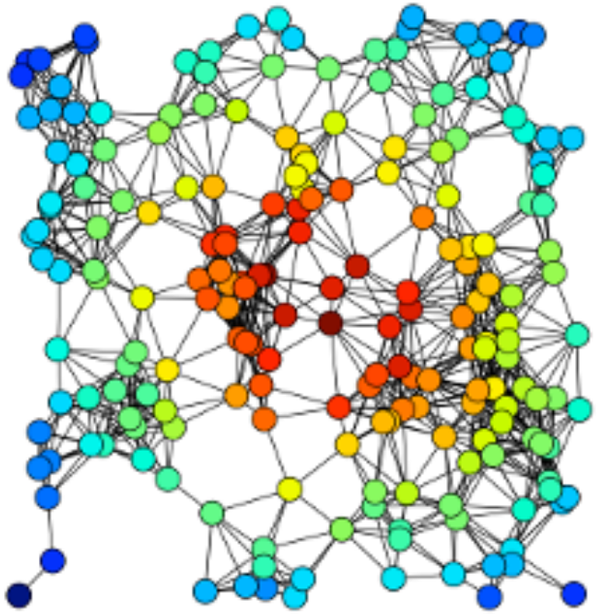
This slide connects eigenvector centrality to PageRank concretely. The random surfer model is intuitive and the power-iteration implementation is computationally attractive: a few sparse matrix-vector updates scale better than all-pairs shortest paths. But PageRank is a prestige-through-links model, and link farms exploit exactly that assumption. This is a good moment to contrast endorsement-based centrality with distance-based measures such as harmonic centrality: they answer different questions and have different computational profiles.

Same Graph, Four Views of Importance



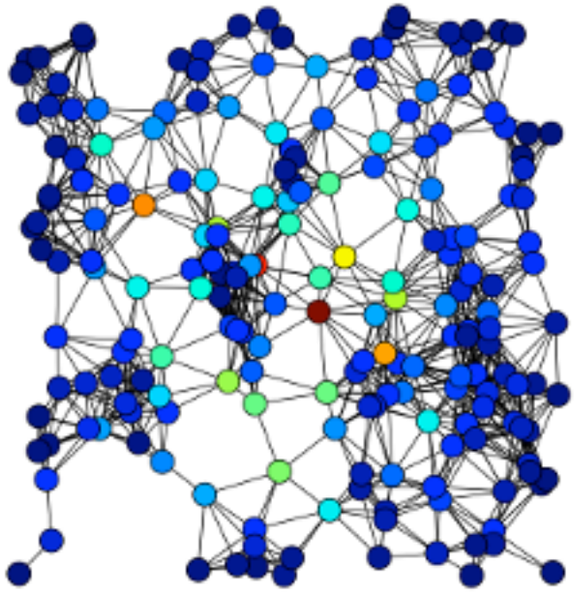
Source: Easley & Kleinberg 2010

Degree



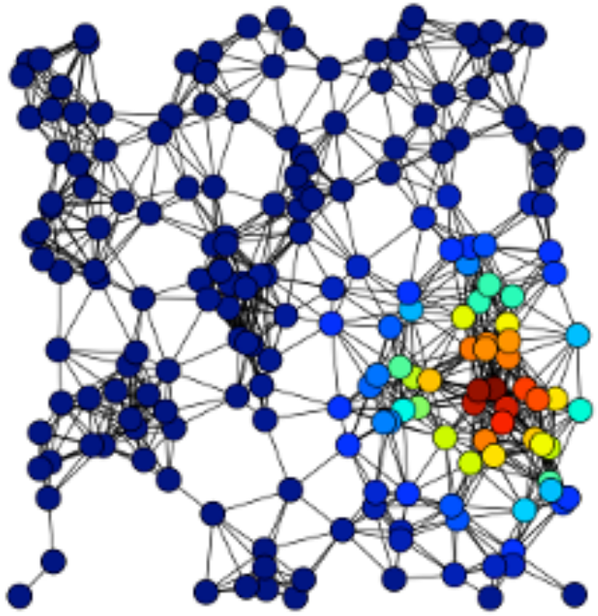
Source: Easley & Kleinberg 2010

Closeness



Source: Easley & Kleinberg 2010

Betweenness



Source: Easley & Kleinberg 2010

Eigenvector

Node size in each panel encodes that panel's centrality value. The same graph yields different "most important" nodes depending on which measure is used — confirming that centrality is always a theory of importance, not a neutral measurement.

Speaker notes

This visualization makes the central conceptual point of the module concrete: there is no single answer to "which node is most important". The bridge node dominates betweenness but not eigenvector; the densely embedded node dominates eigenvector but not betweenness. Choosing a centrality measure is choosing a theory about what kind of influence matters in the system being analyzed.

Centrality Summary

Notation: $n = |V|$ nodes, $m = |E|$ edges, $k =$ power-iteration steps. "One node" means scoring one focal node; "all nodes" means computing all scores. Sorting a complete ranking adds $O(n \log n)$.

Measure	Theory	One node	All nodes
Degree	direct exposure	$O(\deg v)$, often $O(1)$ if stored	$O(n + m)$
Closeness	short-path access	$O(n + m)$ BFS	$O(n(n + m))$
Harmonic	reachable proximity	$O(n + m)$ BFS	$O(n(n + m))$
Betweenness	path brokerage	$O(n(n + m))$ exact	$O(n(n + m))$ with Brandes
Eigenvector / PageRank	recursive prestige	computed as whole vector	$O(k(n + m))$

For closeness and harmonic centrality, moving from one focal node to all nodes multiplies the shortest-path computation by about n. For exact betweenness, however, even one focal node already requires all-source shortest-path dependency information, so Brandes computes all nodes for essentially the same asymptotic cost.

> With positive weighted edges, replace BFS by Dijkstra: one source is roughly $O(m + n \log n)$ and all sources $O(nm + n^2 \log n)$. Weighted Brandes is commonly stated with the same $O(nm + n^2 \log n)$ distance-search term.

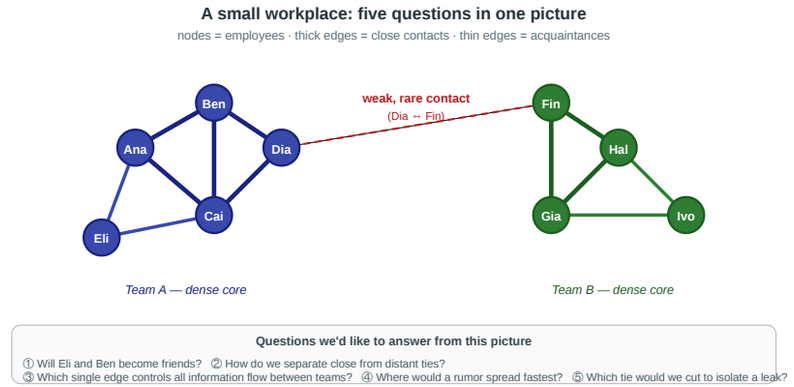
Speaker notes

This slide should prevent the common misunderstanding that a centrality formula has one universal cost. Use the general unweighted graph bounds: BFS is $O(n+m)$, not just $O(m)$, because disconnected or sparse graphs still require node bookkeeping. For closeness and harmonic centrality, one focal node is one shortest-path search, while all nodes require one search per source. Exact betweenness is global: Brandes computes all scores in $O(n(n+m))$ for unweighted graphs, and computing one exact focal score does not usually give the same simple savings as closeness. Eigenvector and PageRank are naturally vector computations.

Back to the Workplace

In the workplace figure, each measure picks a different winner:

- **Degree** → Ana (or Ben): most direct contacts inside team A's core
- **Closeness** → Dia: smallest average distance to everyone, because she sits on the only bridge
- **Betweenness** → Dia *and* Fin: every shortest path between the teams passes through both
- **Eigenvector** → Ana or Ben: connected to densely-connected others, which amplifies recursively



Speaker notes

Running through the four centralities on the same small concrete graph makes the pluralism tangible. The crucial insight: the "most important" answer depends on which theory of importance you adopt. Dia wins on betweenness because she is the only broker, but she loses on eigenvector because team B's neighbors are less densely connected than team A's. In the workplace figure there is no single "most important" node — there is only a most important node *for a given question*.

Takeaway

> Centrality is always tied to a theory of influence, access, or control. Choosing a measure is choosing a theory — not collecting an objective fact.

Quick Check

For each scenario, decide which centrality measure is most appropriate and justify briefly.

1. You want to identify the most influential gossip in a small village.
2. You want to find emergency-response hubs that can reach all districts within minimum time.
3. You want to detect which router, if attacked, would slow internet traffic between continents most.
4. You want to find the most prestigious researcher in a citation network.

Speaker notes

The four scenarios are deliberately matched one-to-one with the four measures. Students who answer "degree" or "betweenness" to all four miss the conceptual point: each scenario embeds a different theory of what "important" means, and the right measure is the one that matches the theory.

Quick Check

For each scenario, decide which centrality measure is most appropriate and justify briefly.

1. You want to identify the most influential gossip in a small village.
2. You want to find emergency-response hubs that can reach all districts within minimum time.
3. You want to detect which router, if attacked, would slow internet traffic between continents most.
4. You want to find the most prestigious researcher in a citation network.

Solution:

1. **Degree** — direct contacts matter.
2. **Closeness** — minimum average distance to districts.
3. **Betweenness** — router lies on intercontinental paths.
4. **Eigenvector / PageRank** — prestige flows from being cited by prestigious others.

What Counts as a Community?

Guiding Question: Can we give "community" a mathematical definition precise enough that an algorithm could find one?

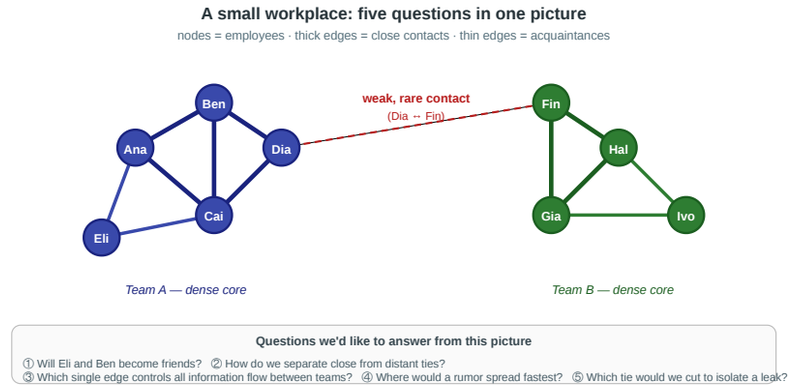
Speaker notes

Communities are intuitively obvious in a drawing but surprisingly hard to define formally. The definition determines what algorithms can detect and what they will miss. This module introduces modularity as the canonical answer — and then shows that the canonical answer is computationally out of reach, exactly as MaxSTC was in Lecture 03.

Working Definition

Community (informal). A subset of nodes $S \subseteq V$ with

- relatively many edges *inside* S
- relatively few edges *leaving* S
- density meaningfully higher than the surrounding graph.



Different formal definitions instantiate this intuition differently. The most influential is **modularity**, which compares observed internal density against a random null model.

Speaker notes

The informal definition is intentional: it gives the structural criterion without committing to a specific objective function. Modularity, conductance, and density-based definitions all satisfy the informal criterion but operationalize it differently. The pluralism is a feature — different applications need different sharpnesses of community boundary — but for the rest of the lecture we focus on modularity because it is the single most widely used objective.

Modularity — The Canonical Objective

Modularity Q of a partition $C_1, \ldots, C_k$ of a graph with m edges and adjacency matrix A:

Pairwise form

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

Equivalent grouped form

$$Q = \sum_{c} \left(\frac{l_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right)$$

where k_i is node degree, $\delta(c_i, c_j) = 1$ for same-community pairs, l_c is the number of internal edges in community c, and d_c is the sum of degrees in c.

Interpretation. For every within-community pair, compare the *observed* edge A_{ij} to the *expected* edge density $k_i k_j / 2m$ under a random rewiring that preserves degrees (the **configuration model**). Q is the total surplus of within-community edges over chance.

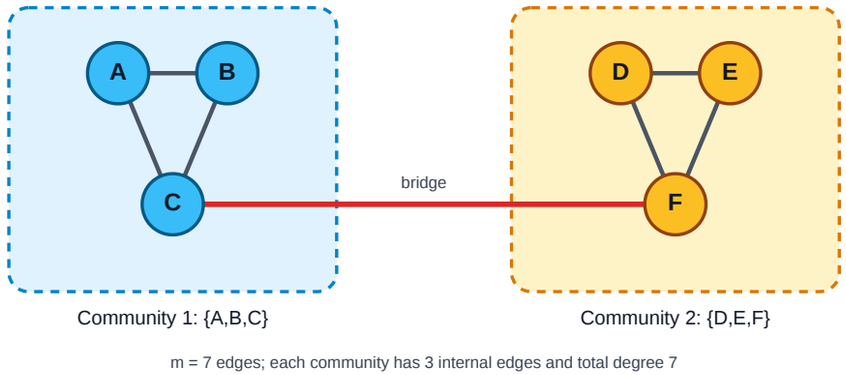
Q ranges roughly in $[-\frac{1}{2}, 1]$. Values in $[0.3, 0.7]$ typically indicate significant community structure; $Q \approx 0$ means no more internal density than expected by chance.

Speaker notes

Modularity is the single most important concept in this lecture. It converts the vague notion of "community" into a number we can optimize. The configuration-model null is what makes it statistical rather than arbitrary: instead of asking "are there many edges inside this group?" it asks "are there *more* edges inside this group than we would expect if we kept each node's degree but rewired everything at random?". That reframing is precisely why modularity can distinguish real communities from high-degree hubs whose connections merely happen to concentrate.

Modularity — Worked Example

Modularity is calculated for the **full graph** together with a **given partition**. Here all six nodes are assigned to two communities; then Q asks whether this complete partition has more internal edges than expected by chance.



For each community:

- internal edges: $l_c = 3$
- total degree: $d_c = 2 + 2 + 3 = 7$
- full graph edges: $m = 7$

$$Q = 2\left(\frac{3}{7} - \left(\frac{7}{14}\right)^2\right) = 2(0.429 - 0.25) \approx 0.36$$

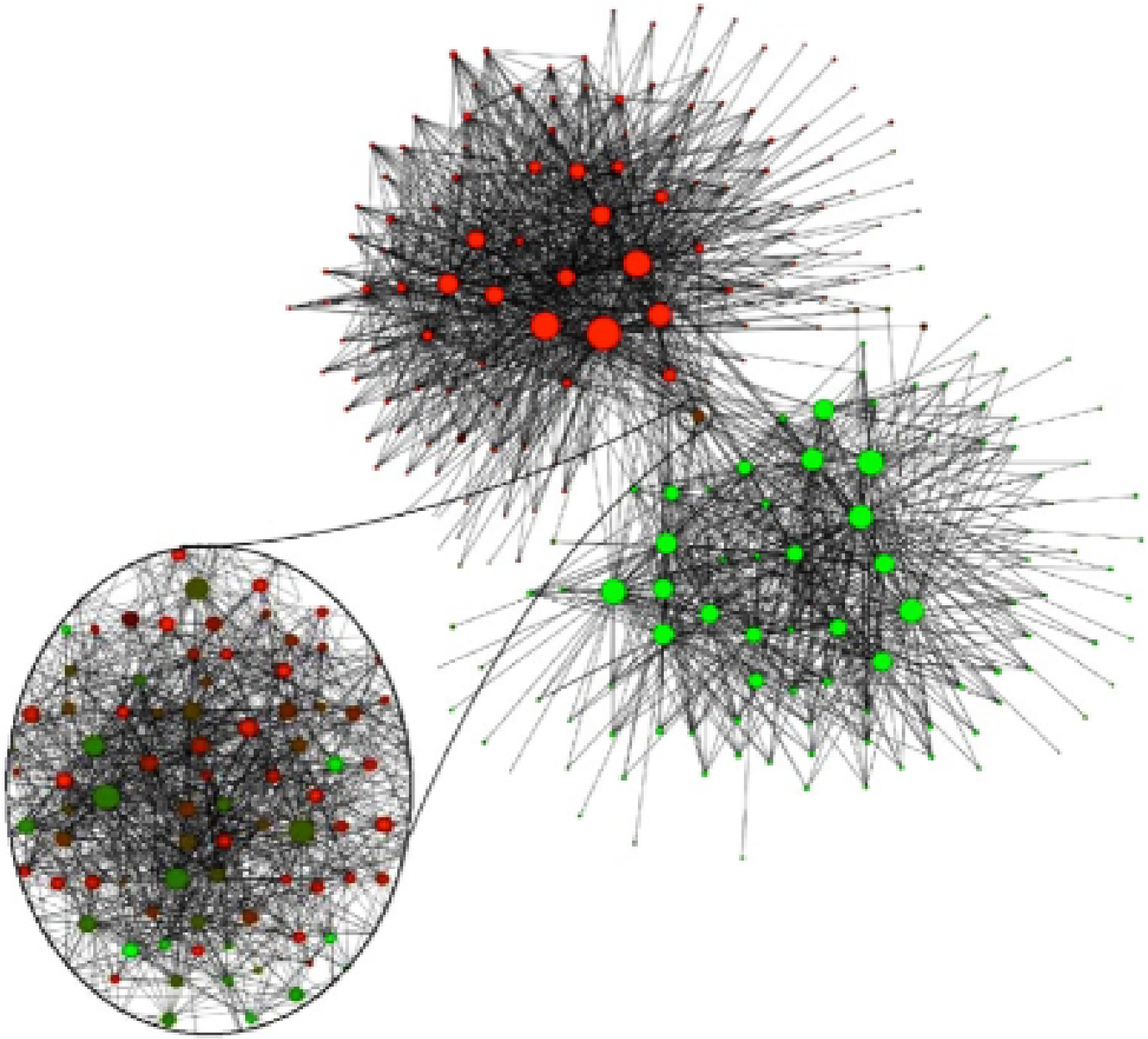
Interpretation: the partition has more within-community edges than expected under degree-preserving random rewiring, so Q is positive.

Given partition: $C_1 = A, B, C$ and $C_2 = D, E, F$. The bridge is the only between-community edge.

Speaker notes

This is the smallest useful modularity example: two dense triangles and one bridge. The grouped formula is just the double-sum modularity formula collected by community; introduce it explicitly so "community form" is not a new unexplained concept. The arithmetic is correct: each triangle has three internal edges, and the bridge gives one node in each triangle degree three, so the total degree per triangle is seven. The positive Q value says this two-community partition captures real excess internal density.

Example Case — Belgian Phone Calls



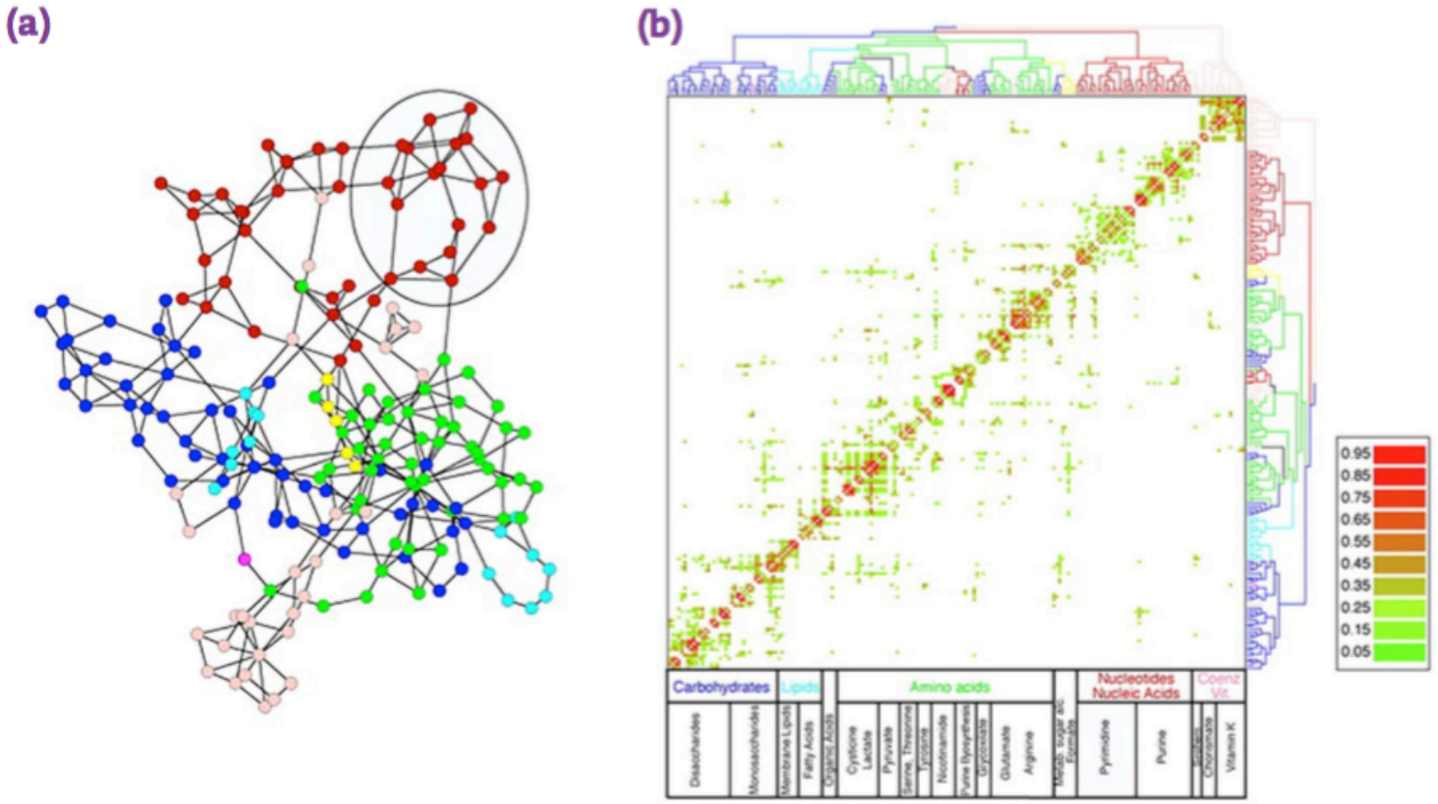
Nodes are phone users or regions; edges are call relationships. Community detection recovers dense calling regions, and the strongest split aligns with Belgium's Flemish-Walloon linguistic divide — a social boundary inferred from communication structure, not supplied as a label.

Source: Easley & Kleinberg 2010

Speaker notes

Use this as the social-network example. The important point is not that the algorithm knows language. It only sees calls. The linguistic boundary appears because people communicate more within language communities than across them.

Example Case — Metabolic Networks



Source: Easley & Kleinberg 2010

Nodes are metabolites or reactions; edges represent biochemical transformations. Communities correspond to functional modules: groups of reactions that interact densely because they participate in the same pathway or cellular process.

Speaker notes

Use this as the biological-network example. The same structural definition of community now means functional organization rather than social identity. This helps students see why modularity is useful across domains: dense internal connectivity can signal linguistic groups, biological pathways, or technical subsystems depending on what the nodes and edges mean.

The Catch — Modularity Maximization Is NP-Hard

> **Result (Brandes et al., 2008).** Finding the partition that maximizes Q on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

Speaker notes

This is the slide where the L03–L04 narrative arc closes. In L03, we had a clean theoretical construct (STC labeling) whose exact recovery (MaxSTC) was NP-hard, so we fell back on clustering coefficient and neighborhood overlap as polynomial-time proxies. In L04, we have a clean theoretical construct (modularity-optimal partition) whose exact recovery is also NP-hard, and we will fall back on Girvan–Newman and Louvain as polynomial-time proxies. Same shape, different level of the graph.

The Catch — Modularity Maximization Is NP-Hard

> **Result (Brandes et al., 2008).** Finding the partition that maximizes Q on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

Sound familiar? This is the exact same situation as **MaxSTC** in Lecture 03: a theoretically clean objective that is computationally out of reach on real data.

The Catch — Modularity Maximization Is NP-Hard

> **Result (Brandes et al., 2008).** Finding the partition that maximizes Q on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

Sound familiar? This is the exact same situation as **MaxSTC** in Lecture 03: a theoretically clean objective that is computationally out of reach on real data.

Consequence. The rest of the module introduces two families of polynomial-time *heuristics* that find partitions with high Q without guaranteeing the global maximum:

- **divisive** (Girvan–Newman): peel apart high-betweenness edges
- **agglomerative** (Louvain, Leiden): greedily merge nodes that increase Q

Study — International Trade Communities

Cho, Lee & Kim (2023) model international trade as a network of countries connected by trade relationships and detect **trade communities at multiple resolutions** (Cho et al. 2023).

Network formation. Nodes are countries; weighted edges encode trade relationships. Strong edges form when countries exchange large volumes or are tightly embedded in the same trade system.

Question. Does world trade split into one stable map of blocs, or into nested communities that depend on resolution?

Finding. A single community scale can hide structure: large blocs contain smaller regional or sectoral sub-blocs. Economic communities are not only geographic; they also reflect trade intensity, specialization, agreements, and supply-chain structure.

Panels

$\Phi^{}$ *is the* ***cooccurrence threshold****: countries are grouped only if they appear together in the same community in at least that share of multiresolution runs. Higher* $\Phi^{}$ keeps only more stable links.

- **a** $\Phi^* = 0$: one large trade component
- **b** $\Phi^* = 0.41$: four broad blocs
- **c** $\Phi^* = 0.96$: 16 stable country groups

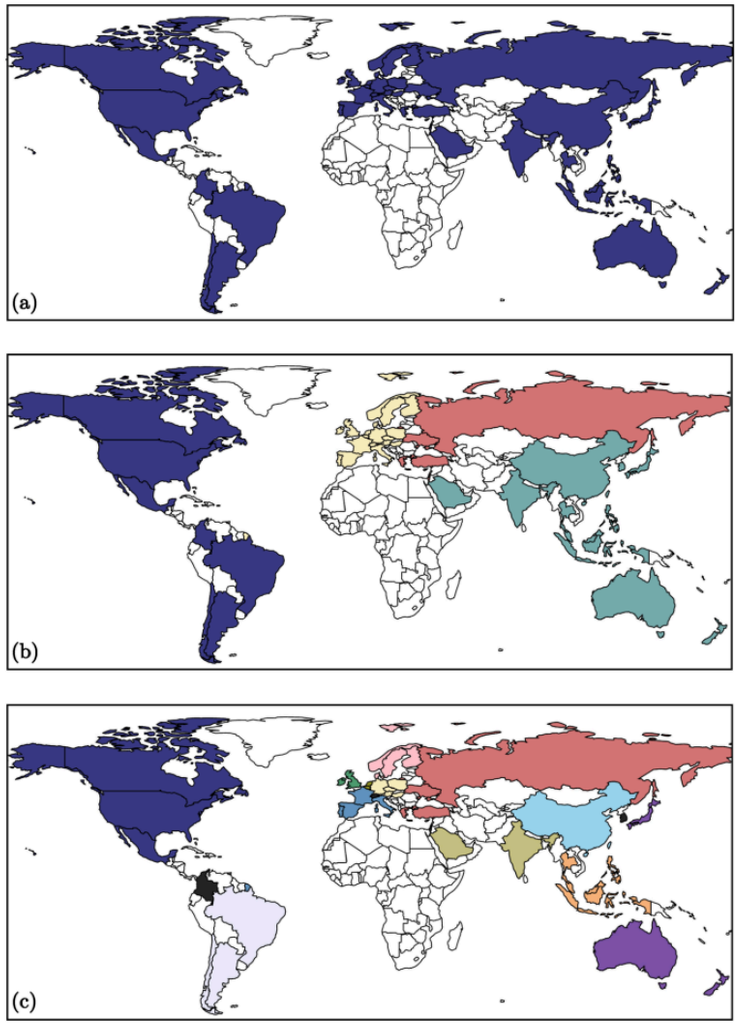


Figure: selected country configurations at cooccurrence thresholds $\Phi^* = 0, 0.41$, and 0.96; reproduced from Fig. 4 in Cho, Lee & Kim (2023), CC BY 4.0.

Speaker notes

Use this slide to change domain from social and biological examples to economic networks. We did not previously have a Cho et al. figure in the local figures directory; this adds Figure 4 from the open-access Scientific Reports article under CC BY 4.0. The key point is methodological: the resolution parameter matters. Trade has nested structure, so one modularity scale is too coarse. Community detection here is not just descriptive; it asks whether the global economy is organized into stable blocs at multiple scales.

Takeaway

Modularity Q is the canonical mathematical definition of "community": excess internal density over a random-graph null model. Maximizing it exactly is NP-hard — so every practical community detection method is a heuristic.

Quick Check

A graph has two dense clusters of 50 nodes each connected by a single edge, plus one small triangle attached to one cluster via a single edge.

1. Roughly how many natural communities does this graph have?
2. Will modularity maximization find all of them?
3. If you asked a balanced 2-partition algorithm to split it into two equal halves, what might go wrong?

Speaker notes

The graph has three natural communities, and modularity maximization should find them on a graph this size — but there is a subtle trap. Modularity has a well-known *resolution limit* (Fortunato & Barthélemy 2007): in large graphs, modularity cannot detect communities smaller than roughly $\sqrt{2m}$ edges. On the small example above the triangle is easy to find, but if the two big clusters grew to 5000 nodes each, the triangle would disappear below the resolution threshold. The balanced partition question is a separate trap: enforcing equal sizes forces the algorithm to either cut through a real community or merge the triangle with the wrong neighbor. Either way, a structural constraint hides real structure.

Quick Check

A graph has two dense clusters of 50 nodes each connected by a single edge, plus one small triangle attached to one cluster via a single edge.

1. Roughly how many natural communities does this graph have?
2. Will modularity maximization find all of them?
3. If you asked a balanced 2-partition algorithm to split it into two equal halves, what might go wrong?

Solution:

1. **Three** natural communities: two dense clusters plus the triangle.
2. On this small graph, likely yes; at larger scale, modularity's **resolution limit** can hide small groups.
3. Balanced 2-partition may cut through a dense cluster or merge the triangle with the wrong side.

Finding Communities — Divisive and Agglomerative

Guiding Question: If modularity maximization is NP-hard, how do we find good partitions in practice?

Speaker notes

This module collects the practical heuristics used to approximate modularity maximization. Two broad families: divisive (Girvan–Newman) works top-down by removing inter-community edges; agglomerative (Louvain, Leiden) works bottom-up by merging nodes that increase modularity. Both produce hierarchies, and both are chosen over exact optimization because they are polynomial-time.

Two Strategies

Divisive (top-down). Start with the whole graph as one cluster. Repeatedly remove edges that lie *between* communities, peeling the graph apart. Example: Girvan–Newman.

Agglomerative (bottom-up). Start with each node as its own cluster. Repeatedly merge the pair whose merge most *increases* modularity. Example: Louvain, Leiden.

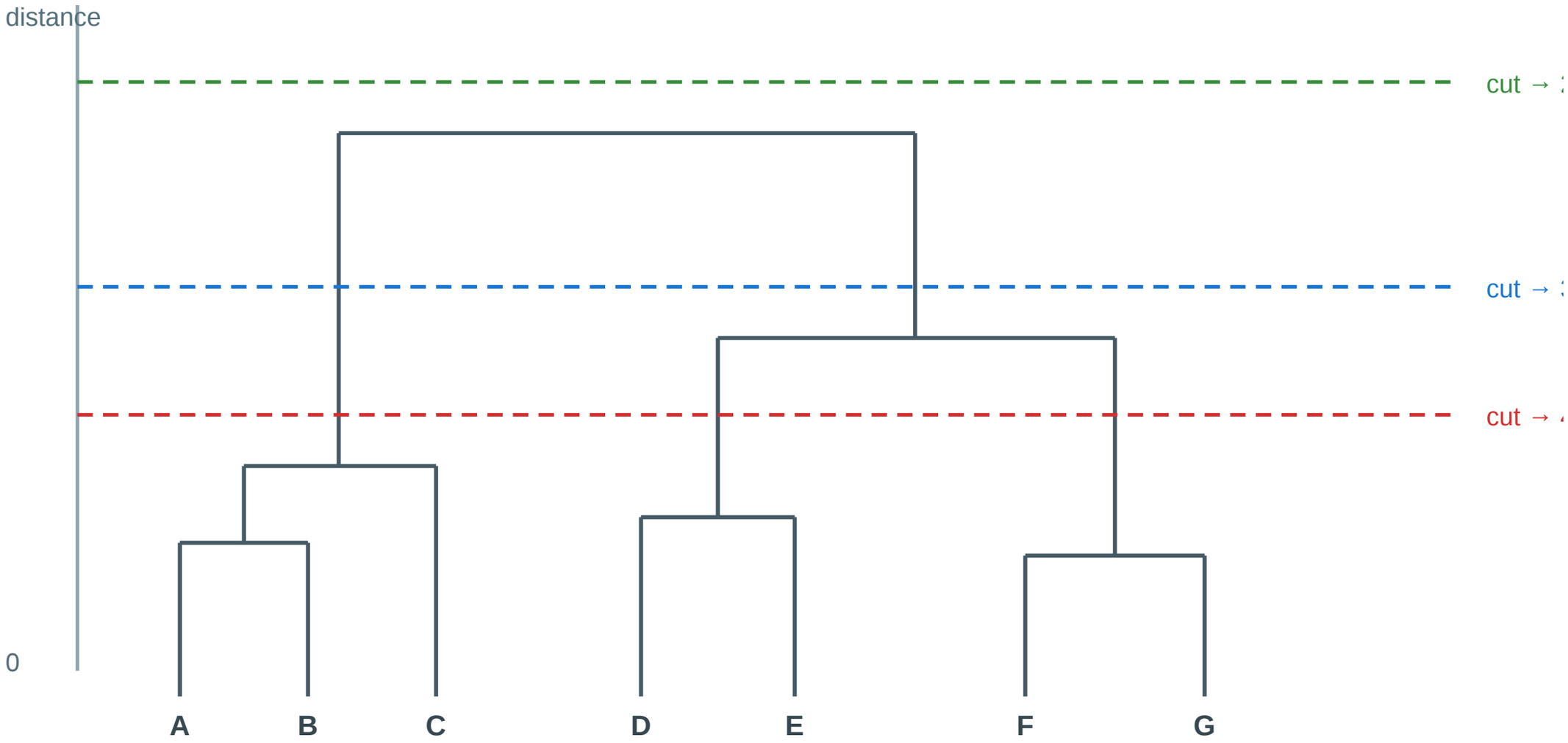
Both strategies produce a hierarchy — a **dendrogram** — rather than a single flat partition. Where to cut the dendrogram is the analyst's modeling decision.

Speaker notes

Divisive methods target the inter-community edges directly, which aligns with the bridge intuition from Lecture 03 — inter-community edges carry many shortest paths, so they have high edge-betweenness. Agglomerative methods are cheaper and currently dominate practice (Louvain is the default in NetworkX and Gephi, Leiden is the modern improvement). Both families produce dendrograms, so the module-level takeaway is the same: algorithms give you a hierarchy; you, the analyst, decide where to cut.

The Dendrogram as Output

Hierarchical clustering: the cutoff is a modeling choice



Each horizontal cut yields a different clustering — the analyst chooses the level

Every agglomerative or divisive run produces a hierarchy like this. Each horizontal cut is a valid clustering; the "right" one depends on the downstream task. Picking the cut with the highest modularity Q is the default rule.

Speaker notes

The dendrogram is the explicit representation of multi-scale community structure. Cutting it at different heights produces different partitions, and each cut is structurally valid. The decision about where to cut is necessarily external to the algorithm — it depends on what the communities will be used for, or on picking the height that maximizes Q.

Girvan–Newman — The Divisive Algorithm

Algorithm: Girvan–Newman

(Input: Graph $G = (V, E)$*)*

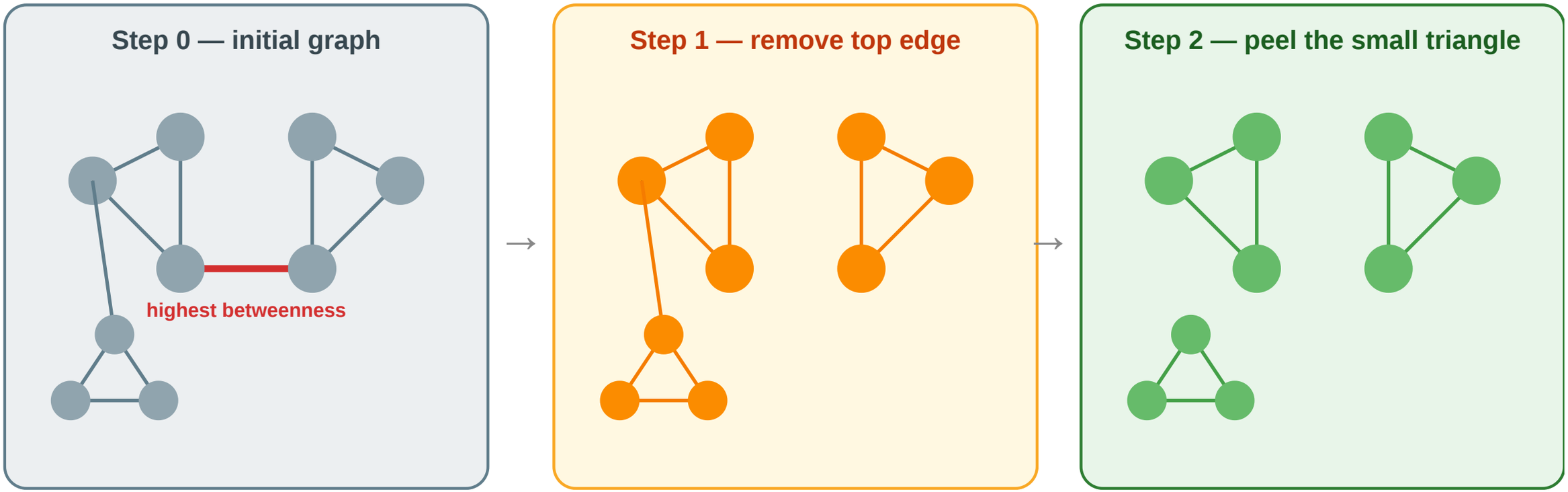
1. Compute the **edge betweenness** of every edge — the fraction of all shortest paths that pass through it.
2. Remove the edge with the **highest** edge betweenness.
3. Recompute edge betweenness on the remaining graph.
4. Repeat until no edges remain — record the partition at each step.
5. Choose the partition that maximizes **modularity** Q.

Edges with high edge-betweenness are exactly the **local bridges** and **weak ties** from Lecture 03 — they carry many shortest paths because all cross-community communication must cross them. Girvan–Newman is an algorithmic version of Granovetter's weak-tie argument.

Speaker notes

The recompute step in line 3 is what makes Girvan–Newman conceptually clean but computationally expensive: edge betweenness must be recalculated after every removal because removing one edge changes the shortest paths used by all the others. The whole algorithm runs in $O(|V| \cdot |E|^2)$ worst case — fine for teaching, impractical for graphs beyond a few thousand nodes. This is why modern practice uses Louvain / Leiden instead.

Girvan–Newman — Worked Example



two main clusters revealed

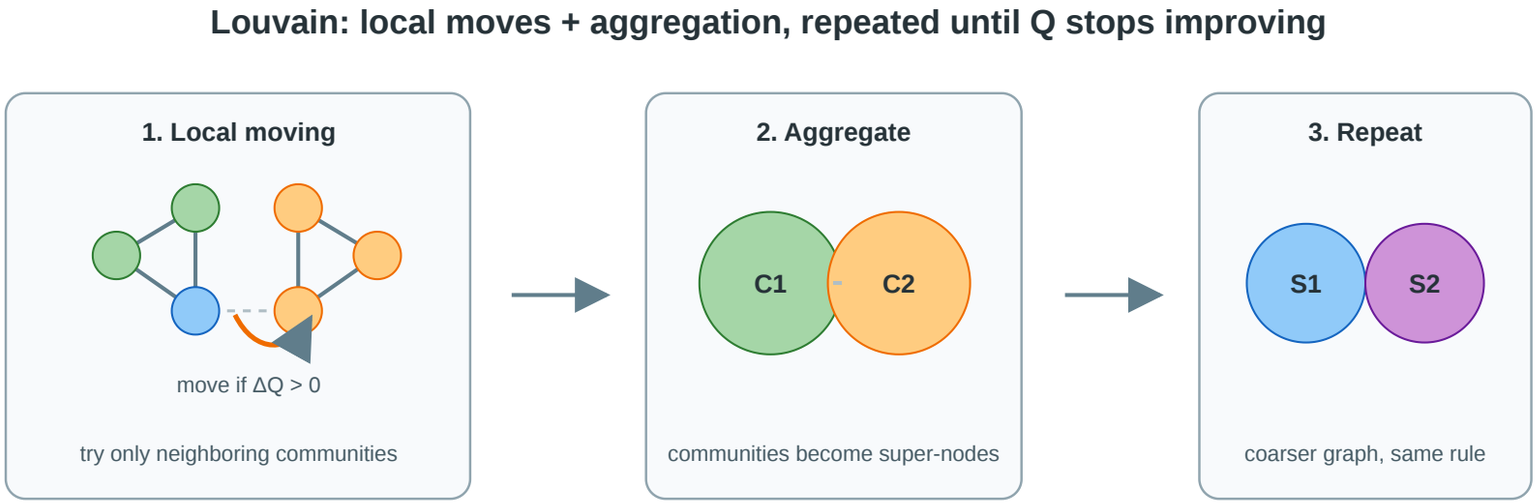
- **Step 0:** the bridge edge has highest betweenness because every path between the two main clusters must cross it.
- **Step 1:** removing that bridge reveals two main clusters; the small triangle remains weakly attached to the left cluster.
- **Step 2:** the next high-betweenness edge is the link to the triangle; removing it isolates the third community.
- **Final partition:** three communities.

Speaker notes

The worked example shows the algorithm's two characteristic moves: detaching a major community boundary, then peeling smaller substructures. Notice that edge-betweenness recomputation is essential — after step 1, the highest-betweenness edge changes because the graph topology has changed.

Louvain — Greedy Modularity in Two Phases

Louvain (Blondel et al., 2008) is a fast greedy heuristic for high-modularity partitions.



No pre-given k: the number of communities emerges from accepted modularity-improving moves and later aggregation levels.

For a node v, Louvain tests the current communities found among v's neighbors, plus staying where it is.

Algorithm Sketch

- **Initialization:** start with $k = n$ communities: every node is its own community.
- **Local moving:** visit node v and move it to another community only if this increases Q.
- **Candidate communities:** for v, test the current communities of v's neighbors, plus staying where it is. At the first pass these candidates are just neighboring single-node communities.
- **Aggregation:** collapse each detected community into a super-node.
- **Repeat:** run local moving again on the smaller graph until modularity no longer improves.
- **No fixed k:** the number of communities is not pre-given; it emerges from the greedy moves and aggregation levels.

Drawbacks: Louvain is greedy, so results can depend on node order and local optima. It inherits modularity's resolution limit and can produce internally weak or disconnected communities

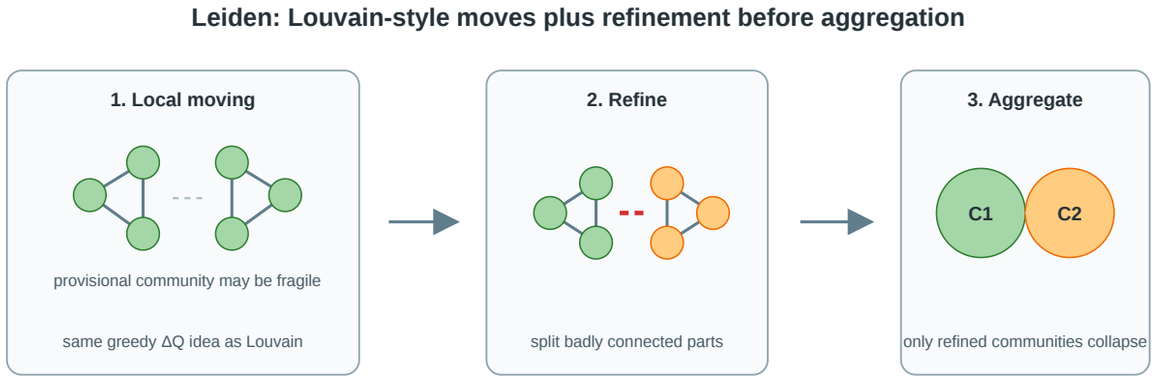
because it aggregates provisional groups without a refinement check.

Speaker notes

Louvain is currently a default community-detection algorithm in Gephi and many network-analysis packages because it avoids all edge-betweenness recomputation. The key intuition is compression: local moves find provisional communities, then aggregation turns those communities into a smaller graph, so the same operation can be repeated at a coarser scale. The "k" question is important: Louvain does not test a pre-given number of communities. At the start k=n, because every node is alone. During local moving, a node only considers the communities of its neighbors; during aggregation, the current k becomes the number of super-nodes in the next level. The final k is therefore an output of the heuristic, not an input.

Leiden — Louvain with Refinement

Leiden (Traag et al., 2019) keeps Louvain's speed idea but adds a refinement phase before aggregation.



Still no pre-given k: Leiden also lets the partition size emerge, but prevents badly connected communities from being aggregated too early. The refinement step gives stronger internal-connectivity guarantees than Louvain while keeping the same scalable compression logic.

Algorithm Sketch

- **Local moving:** first improve Q with Louvain-style node moves.
- **Refinement:** inspect each provisional community before aggregation. Start again with its nodes as small subcommunities, then merge/move only if the subcommunity remains well connected and improves the quality function.
- **Split effect:** if a Louvain community is held together only by a weak bridge, Leiden can keep its parts separate instead of collapsing them into one super-node.
- **Aggregation:** collapse only the refined communities and repeat on the smaller graph.
- **No fixed k:** Leiden also lets k emerge, but it prevents weak provisional groups from being locked in too early.

Both, Louvain and Leiden, are heuristic modularity optimizers: they do not guarantee the global maximum, but they scale to graphs where Girvan-Newman is infeasible. Leiden is usually preferred in research pipelines because it preserves Louvain's scalability while avoiding structurally broken communities.

Speaker notes

Leiden is best taught as "Louvain plus a quality-control step." The flaw is not merely theoretical: after greedy local moves and aggregation, Louvain may label a set of nodes as one community even though parts of that set are not internally connected in a meaningful way. Leiden inserts a refinement phase that checks and splits such communities before aggregation. It is still a heuristic, but it returns communities with stronger internal connectivity guarantees.

Graph Partitioning — Cut-Based Methods

Min-cut / Max-flow (Ford–Fulkerson) Find the smallest set of edges whose removal disconnects two specified groups. Exact in polynomial time — but minimizes raw cut size, not normalized density.

Kernighan–Lin (1970) Local search: initialize a balanced 2-partition at random; iteratively swap node pairs to decrease the cut size. Fast in practice; converges to a local optimum.

Spectral partitioning Compute the Laplacian $L = D - A$ of the graph. The **Fiedler vector** (eigenvector of the second-smallest eigenvalue λ_2) encodes the natural 2-partition: nodes with positive entries go left, negative entries go right.

λ_2 is the **algebraic connectivity**: $\lambda_2 = 0$ means the graph is already disconnected. Spectral methods generalize to k communities by using the k smallest eigenvectors — the basis of **spectral clustering**.

Speaker notes

Cut-based partitioning predates modularity and remains important in chip design, load balancing, and parallel computing — contexts where the exact number of parts is fixed and balance matters more than organic community structure. Min-cut gives the theoretically tight answer for two specific groups but is sensitive to hub nodes (a single high-degree hub connected to both sides will dominate the cut). Kernighan–Lin is the practical engineering heuristic for VLSI layout. Spectral partitioning bridges graph theory and linear algebra: the Fiedler vector carries the global geometry of the graph into a 1D projection that can be thresholded to produce a partition. The connection to graph Laplacians also connects directly to graph neural networks — the spectral view is the foundation of GCNs.

Embedding-Based Community Detection

Node embeddings + clustering

> **node2vec (Grover & Leskovec, 2016)** Random-walk–based embedding: simulate biased random walks on the graph, then train Word2Vec–style skip-gram to produce a low-dimensional vector per node. Communities emerge as clusters in embedding space.

Pipeline: node2vec → k-means / DBSCAN → community labels

Hyperparameter p controls return probability (DFS-like, captures communities), q controls exploration (BFS-like, captures roles). Setting $p \gg q$ favors dense cluster detection.

Graph Neural Networks

> **GNN-based community detection** Train a GNN encoder (e.g. GCN, GraphSAGE) to produce embeddings that maximize an unsupervised objective — modularity, contrastive loss, or cluster assignment entropy. The decoder maps embeddings to community memberships.

Approach	Key idea
GRAPH-BERT	Self-supervised pre-training on subgraphs
DMGI	Contrastive multiplex embeddings
DMoN	Differentiable modularity as loss

GNNs naturally handle **overlapping communities** (soft cluster assignment) and **attributed graphs** (node features + topology jointly).

Speaker notes

Embedding-based community detection represents the current research frontier. node2vec is now a standard baseline — available in the PyTorch Geometric and NetworkX ecosystems — and runs efficiently on graphs with millions of nodes. The intuition is straightforward: if two nodes co-occur frequently in random walks, they are structurally similar and their embeddings will be nearby; k-means on those embeddings recovers communities. GNN-based methods are more powerful because they can use node and edge features alongside topology, handle soft (overlapping) community membership, and be trained end-to-end with application-specific objectives. The key limitation of all embedding approaches: the embedding dimension and clustering hyperparameters (k, distance threshold) must be chosen before running, adding new modeling decisions that modularity-based methods avoid.

Algorithm Comparison

Method	Strategy	Complexity	Strength	Limitation
Min-cut	Cut minimization	e.g. $O(nm)$ for basic max-flow variants	Exact for 2-partition	Ignores density, favors small cut
Kernighan–Lin	Local swap	$O(n^2)$ per pass	Fast for balanced partition	Local optimum, fixed k
Spectral	Laplacian eigenvectors	$O(n^3)$ full / faster sparse approx	Global structure, principled	Slow exact; needs k in advance
Girvan–Newman	Edge betweenness	$O(nm^2)$ worst case	Hierarchy, no k needed	Too slow for large graphs
Louvain / Leiden	Modularity greedy	near-linear in m empirically	Fast, scalable, no k needed	Resolution limit, local opt
node2vec + clustering	Random-walk embed	walks $O(rnl)$ + training/clustering	Scales, flexible	Needs k, walk hyperparams
GNN-based	Neural message passing	often $O(TLmd)$-like	Soft membership, features	Requires training objective

Speaker notes

This table is the key synthesis slide for the module. Use n for nodes, m for edges, r for walks per node, l for walk length, T for epochs/iterations, L for message-passing layers, and d for hidden dimension. Students should be able to match each method to the right use case: Louvain/Leiden for exploratory analysis of large graphs with no prior on k; spectral clustering for medium-sized graphs where you want a principled 2–5 partition; node2vec for graphs with rich structure where embedding locality is meaningful; GNN-based when you have node features or need overlapping communities. The complexity column signals when each method becomes impractical — Girvan–Newman is pedagogically essential but computationally obsolete above ~10k nodes.

Study — Product Space and Development

Hidalgo, Klinger, Barabasi & Hausmann (2007) build a **product-space network**: nodes are exported products, and edges connect products that require similar capabilities (Hidalgo et al. 2007).

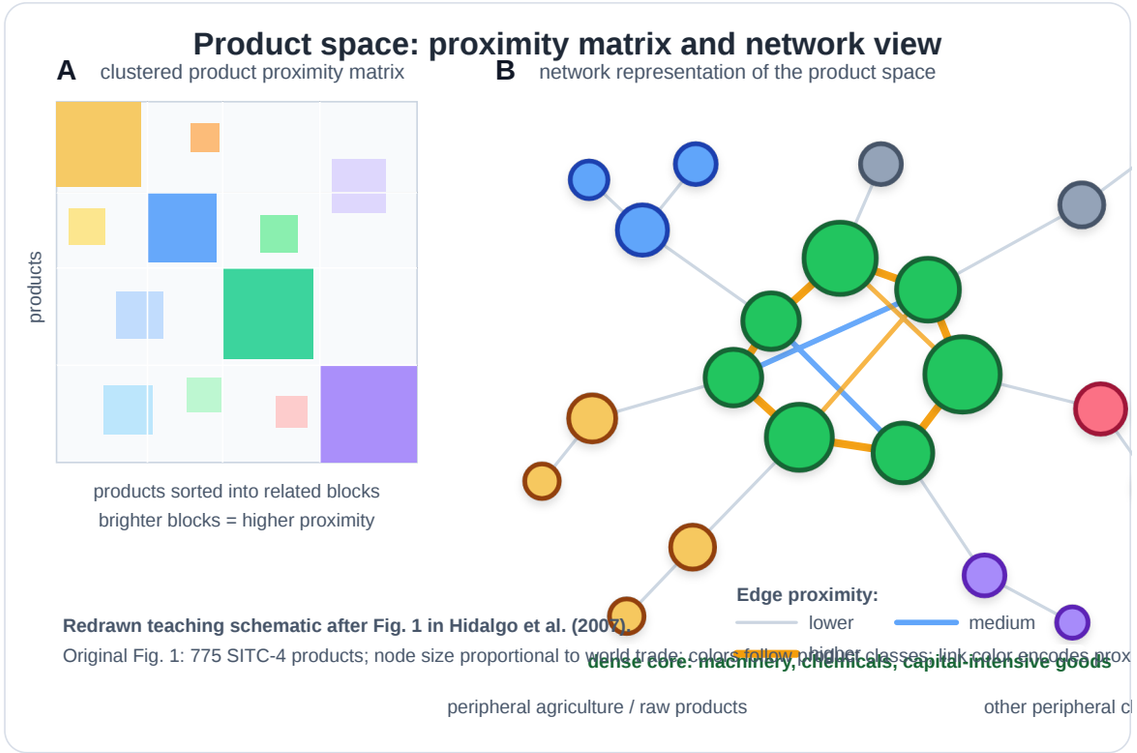
Network formation. Nodes are SITC-4 product classes. A country is linked to a product when it exports it with **revealed comparative advantage** ($RCA > 1$). Two products are similar if countries that export one competitively also tend to export the other competitively:

$$\phi_{ij} = \min P(RCA_i > 1 \mid RCA_j > 1), P(RCA_j > 1 \mid RCA_i > 1).$$

Edges therefore do **not** mean product-to-product trade. They reveal shared capabilities, knowledge, institutions, inputs, or infrastructure.

Question. Why can some countries diversify into sophisticated products while others remain stuck in low-complexity export baskets?

Finding. Sophisticated products sit in a dense core; less sophisticated products often lie in sparse peripheral regions. Countries diversify by moving to nearby products, so peripheral position limits feasible development paths.



Speaker notes

This is not just another social-community example. The network is an economic opportunity map. The community/core-periphery structure tells us which products are close enough for a country to plausibly enter next. It also shows why network position can constrain growth.

Takeaway

Divisive and agglomerative heuristics approximate the NP-hard modularity optimum in polynomial time. Girvan–Newman is the didactic divisive method; Louvain and Leiden are the scalable agglomerative methods used in practice. Graph partitioning (min-cut, spectral) targets fixed-k balanced splits; embedding-based methods (node2vec, GNNs) detect communities from representation space and can handle overlapping memberships and node features.

Quick Check

You run Girvan–Newman on a small graph and get a dendrogram with a maximum Q at the cut that produces 4 communities. You then run Louvain on the same graph and it reports 3 communities with a slightly lower Q.

1. Which answer is "correct"?
2. What could explain the discrepancy?

Speaker notes

Neither is correct in an absolute sense — both are heuristics approximating an NP-hard optimum. Girvan–Newman's divisive peeling can find finer structure because it tests every possible cut, but its higher Q might be fragile to noise. Louvain's greedy merges can get stuck in local optima and miss small communities. The honest answer: both are reasonable, and the discrepancy itself is evidence that the solution is not unique — the student should report both and discuss the tradeoff.

Quick Check

You run Girvan–Newman on a small graph and get a dendrogram with a maximum Q at the cut that produces 4 communities. You then run Louvain on the same graph and it reports 3 communities with a slightly lower Q.

1. Which answer is "correct"?
2. What could explain the discrepancy?

Solution: Neither is absolutely correct. Both are heuristics approximating an NP-hard objective. Louvain may get stuck in a local optimum; Girvan–Newman may expose finer but noisy structure; modularity has a resolution limit; the graph may have multiple near-optimal partitions.

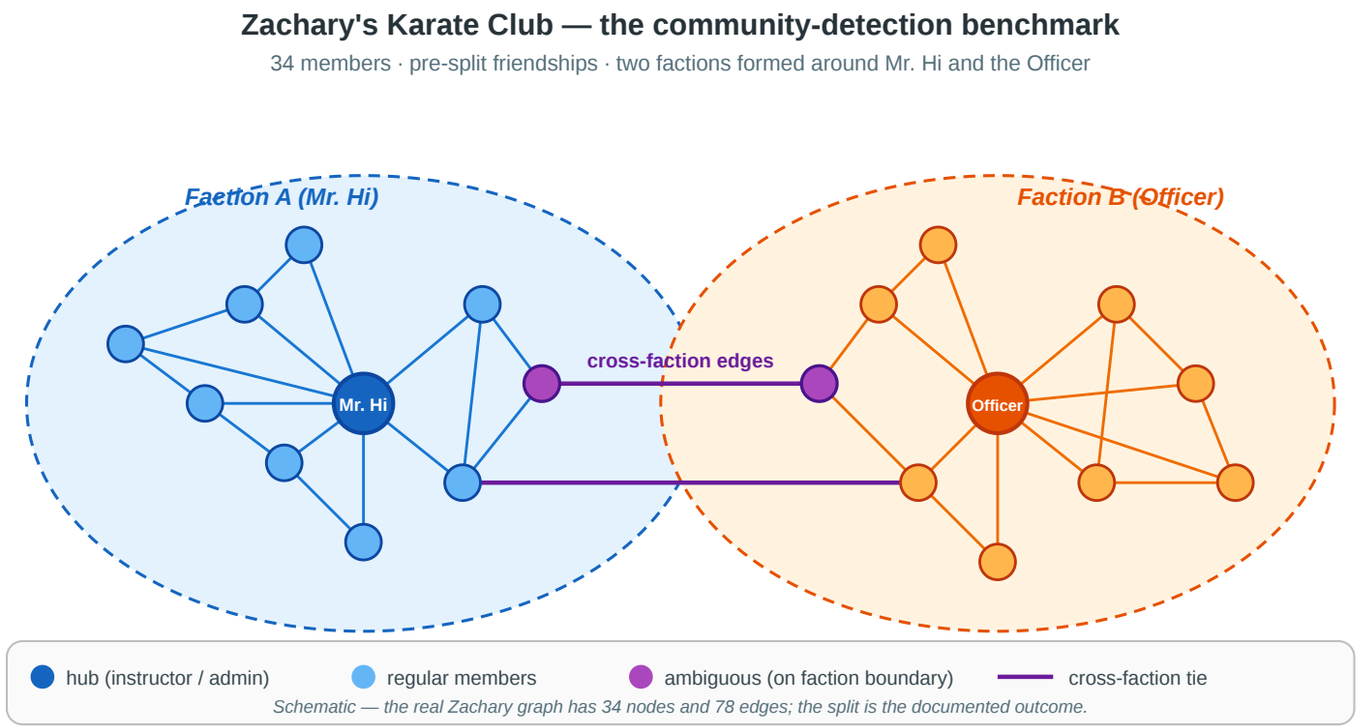
Empirical Anchor — Zachary's Karate Club

Guiding Question: Do community-detection heuristics actually recover real social groups — and how do we know?

Speaker notes

Zachary's karate club is the canonical validation dataset for community detection. It is a small, real social network with a documented ground-truth split, which makes it one of the very few graphs where we can actually check whether an algorithm "got the right answer". It has become so iconic in network science that the first speaker at a conference to mention it in a talk traditionally wins the "Zachary Karate Club Club" prize.

The Story



Reference. Zachary (1977), *An Information Flow Model for Conflict and Fission in Small Groups*, J. Anthropological Research 33, 452–473.

Wayne Zachary observed a university karate club for two years. A conflict arose between the instructor (**Mr. Hi**) and the chief administrator (**Officer**), and the club eventually **split in two** — with each member following one or the other.

Speaker notes

The crucial detail: Zachary recorded the network of *friendships* between members *before* the split happened. The ground truth (who joined which faction) was the natural outcome of the conflict, not an analyst's label. That means we can run any community-detection algorithm on the pre-split graph and directly check how many members it assigns to the correct faction.

What the Algorithms Find

Method	Typical result
Girvan–Newman	Recovers the two factions with **1 misclassified** node
Louvain / Leiden	Recovers the two factions; typically finds **4 sub-groups** at max Q
Max-modularity (exact on this small graph)	Finds the same 4 sub-groups nested inside the 2 factions

The ~1-node misclassification is structurally meaningful: the node in question genuinely sat on the boundary between the two factions during the conflict period, and which side they joined was a close call even to Zachary himself.

Speaker notes

The karate-club result is a success story and a cautionary tale at the same time. Success: the heuristics find the real split from the friendship network alone, which is strong evidence that community structure in social networks is a real phenomenon and not a statistical mirage. Cautionary: the exact modularity maximum finds four sub-groups, not two — modularity's "best" answer disagrees with the ground-truth binary split, because internally the two factions each contain two tightly-connected sub-cliques. The algorithm is not wrong; it is answering a slightly different question.

What the Study Could Not See

Even in this iconic success case, the empirical validation has blind spots that parallel Lecture 03's gap between construct and measurement:

1. Modularity resolution limit (Fortunato & Barthélemy 2007). On the 34-node karate graph the exact max-Q partition has **4** communities, not **2**. The ground-truth split is not even the modularity optimum — the algorithm is answering "which partition maximizes Q", not "which partition matches the real factions".

2. Binary ground truth. Zachary's data records which faction each member *eventually* joined — but not the *strength* of their preference, their uncertainty during the conflict, or whether a third "neutral" group briefly existed. The true social structure may have been continuous or overlapping.

3. Static snapshot. The friendships were recorded at a single time. The dynamics of the conflict — who influenced whom, when someone shifted — are invisible to the algorithm.

Speaker notes

This slide is the L04 analogue of "What the Kossinets-Watts Study Could Not See" in L03. The parallel is intentional: in L03 the binary email data hid tie strength; in L04 the binary faction label hides community overlap and dynamics. Both are reminders that the empirical benchmark is always narrower than the theoretical construct we are trying to validate.

Takeaway

Zachary's karate club shows that community-detection heuristics recover real social structure from graph topology alone — but also that even the "canonical" answer has blind spots: modularity's resolution limit, binary ground truth, and static snapshots all hide real structure the algorithm cannot see.

Wrap-Up — Closing the Modeling Loop

Across Lectures 03 and 04 we have traversed the full modeling/measurement loop twice:

Level	Theoretical ideal	NP-hard recovery	Polynomial proxy	Empirical test
Edges (L03)	Strong/weak labeling	MaxSTC	Clustering coeff., overlap	Kossinets–Watts email
Nodes / Groups (L04)	Modularity-optimal partition	Modularity maximization	Girvan–Newman, Louvain, Leiden	Zachary karate club

Every concept in the two lectures occupies one of those four cells.

Reading

Chapter 3.6 and Chapter 9 of Easley & Kleinberg (2010) cover community structure, brokerage, and centrality. Newman (2010), Chapter 7, gives the formal treatment of centrality measures. For modularity and Louvain, see Blondel et al. (2008) and Traag et al. (2019) for Leiden.

Speaker notes

The closing table is the summary of the L03–L04 arc: two levels of structural analysis, same four-step loop. Lecture 05 will break this pattern by asking a different kind of question: instead of modeling structure, it asks *why* structure forms — when similarity causes friendship, when friendship causes similarity, and how social contexts shape the networks that emerge from them. Homophily, selection, and affiliation networks come next.

Image Credits & References

Barab'asi, Albert-L'aszl'o and Albert, R'eka (1999). Emergence of Scaling in Random Networks. *Science*.

Bonacich, Phillip (1987). Power and Centrality: A Family of Measures. *American Journal of Sociology*.

Burt, Ronald S. (1992). Structural Holes: The Social Structure of Competition. *Harvard University Press*.

Cho, Wonguk and Lee, Daekyung and Kim, Beom Jun (2023). A Multiresolution Framework for the Analysis of Community Structure in International Trade Networks. *Scientific Reports*.

Easley, David and Kleinberg, Jon (2010). Networks, Crowds, and Markets: Reasoning about a Highly Connected World. *Cambridge University Press*. https://www.cs.cornell.edu/home/kleinber/networks-book/

Hidalgo, C'esar A. and Klinger, Bailey and Barab'asi, Albert-L'aszl'o and Hausmann, Ricardo (2007). The Product Space Conditions the Development of Nations. *Science*.